Práctica 4: Logística del Valle

Universidad del Valle de Guatemala · CC2017 Modelación y Simulación · Ciclo 2, 2026

Javier Lopez, 23415

22 de octubre de 2026

Introducción

Logística del Valle es una empresa que distribuye productos de farmacia desde una bodega (N01) hacia cuatro zonas de entrega: N08, N19, N34 y N40. Los conductores usan rutas que aprendieron hace años y la gerencia quiere saber si todavía convienen, y si los 8 vehículos que tiene la empresa alcanzan para las temporadas de más demanda (Semana Santa y Navidad).

Para responder esto construimos el grafo de la red a partir de la matriz de tiempos del anexo A, revisamos si las rutas de las hojas de despacho se pueden recorrer en los dos sentidos y buscamos rutas más rápidas con el algoritmo de Dijkstra. Después hicimos una simulación de eventos discretos de un día de operación para comparar las rutas actuales contra las del algoritmo con distintos niveles de demanda y tamaños de flota.

En resumen, encontramos que las rutas actuales sí son válidas, pero casi ninguna es la más corta. Al cambiarlas, el tiempo promedio de un viaje de ida y vuelta baja de 113.5 a 79.5 minutos y, en un día normal, el porcentaje de entregas a tiempo sube de 57 % a 98 % con la misma flota. También desarrollamos una aplicación en Python (Streamlit) donde se pueden cambiar los parámetros y simular cierres de calles.

Formulación

Objetivo general

Evaluar si las rutas y la flota actuales de Logística del Valle permiten hacer entregas puntuales (en 75 minutos o menos) en los escenarios normal, Semana Santa y Navidad, usando un grafo dirigido y una simulación de eventos discretos, y medir cuánto mejora la operación si se usan rutas de menor tiempo.

Objetivos específicos

1. Construir el grafo dirigido de los 40 puntos a partir de la matriz de tiempos y comprobar si las rutas registradas se pueden recorrer de ida y de regreso.
2. Implementar un algoritmo de ruta más corta y comparar, para cada destino, la ruta actual con al menos dos alternativas, reportando la ida y el regreso por separado.
3. Simular la operación diaria con las dos políticas de rutas usando los mismos pedidos, y comparar puntualidad, espera, tiempo de entrega, utilización de vehículos y tiempo extra.
4. Encontrar a partir de cuántos vehículos la flota deja de ser un problema en cada escenario y ver cómo afectan los cierres o la congestión de algunos tramos.

Datos proporcionados

Dato	Valor
Red	Matriz de adyacencia de 40 x 40 en minutos; celda vacía significa que no hay conexión directa; puede ser asimétrica
Bodega y destinos	N01; N08, N19, N34 y N40, cada uno con la misma proporción de pedidos
Rutas registradas	Cuatro rutas de ida; el regreso es la misma ruta invertida
Recepción de pedidos	8 horas; 4 pedidos/h en día normal, 6 en Semana Santa y 10 en Navidad (promedios)
Flota	8 vehículos, un pedido por viaje, deben regresar a N01 antes del siguiente
Cola	Por orden de llegada
Puntualidad	Entrega en 75 min o menos desde la solicitud, contando la espera

Decisiones a evaluar

Las decisiones que estudiamos son tres: seguir con las rutas actuales o cambiar a las que calcula Dijkstra (con ida y regreso calculados por separado), cuántos vehículos tener (probamos de 4 a 24) y cómo reaccionar si se cierra un tramo o aumenta su tiempo.

Supuestos del modelo

Además de los datos del enunciado, asumimos lo siguiente:

- Los pedidos llegan según un proceso de Poisson, o sea que el tiempo entre llegadas es exponencial con media de 60/λ minutos.
- El destino de cada pedido se escoge al azar con probabilidad de 1/4 para cada zona.
- En el caso base los tiempos de viaje son exactamente los de la matriz y la descarga en el destino dura 0 minutos. En un análisis de sensibilidad agregamos 5 minutos de descarga y variabilidad en el tráfico.
- Todos los vehículos son iguales, empiezan en la bodega y no tienen fallas ni descansos.
- Solo se reciben pedidos durante las 8 horas, pero la simulación continúa hasta que el último vehículo regresa a N01.
- Si hay un vehículo libre, sale de inmediato con el primer pedido de la cola; no se juntan pedidos.
- Si se cierra un tramo de la ruta actual, el conductor sigue su ruta hasta el punto antes del cierre y de ahí toma el camino más corto.

Construcción del grafo

Cada celda con un valor (i, j) de la matriz la interpretamos como un arco dirigido de i hacia j, con un peso igual a los minutos de viaje. Las celdas vacías y la diagonal (que tiene 0) no generan arcos. Con esto obtuvimos un grafo dirigido y ponderado de 40 nodos y 144 arcos. Comprobamos que es fuertemente conexo, es decir, desde cualquier punto se puede llegar a cualquier otro.

Como el enunciado no trae un mapa, para dibujar el grafo acomodamos los nodos en una cuadrícula de 5 filas y 8 columnas según su número (N01 a N08 en la primera fila, N09 a N16 en la segunda, y así). Esta forma tiene sentido porque en la matriz aparecen conexiones como N01-N09, N02-N10 y N08-N16, que quedan como tramos verticales.

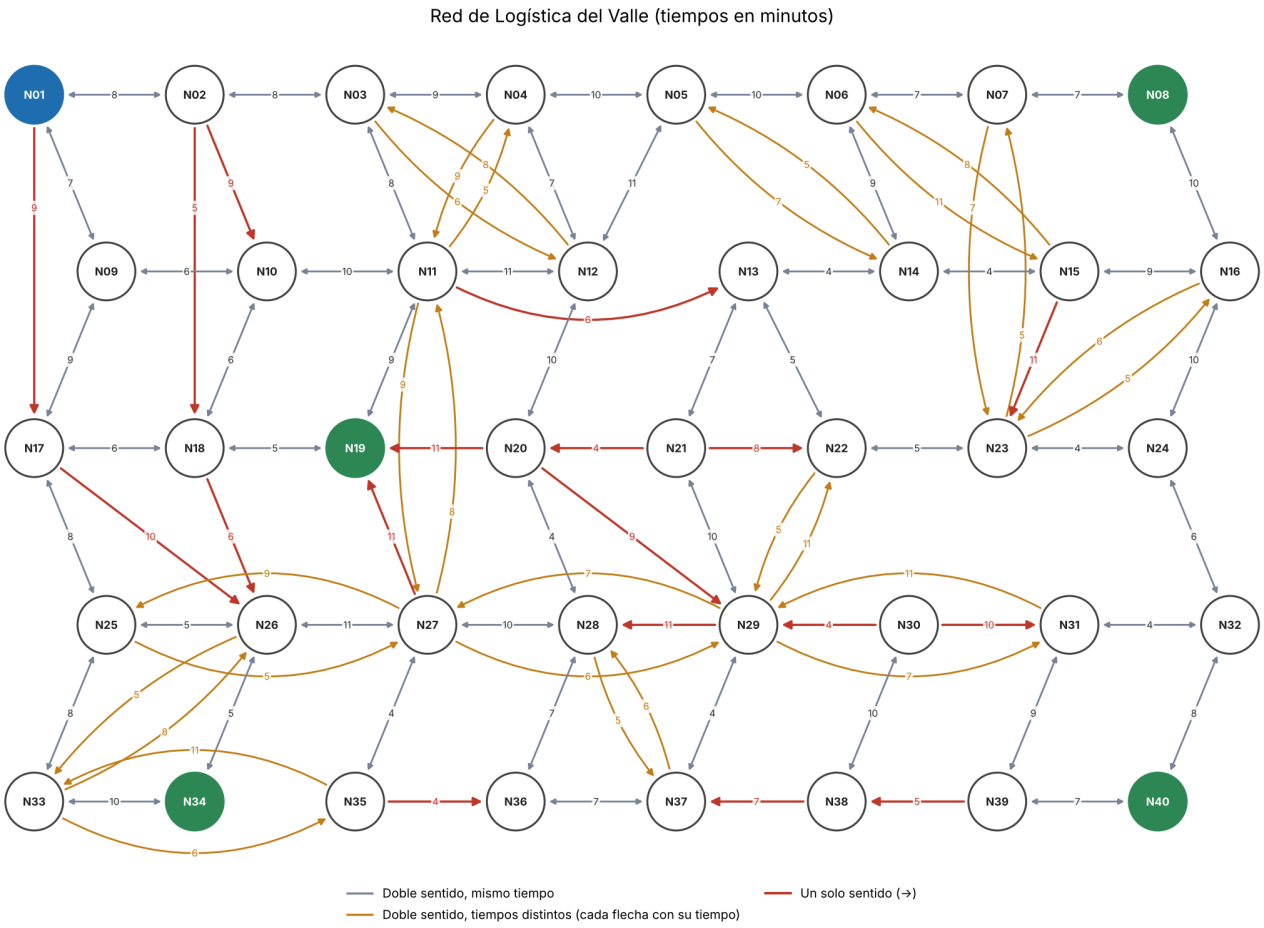


Figura 1. Grafo de la red con los tiempos de viaje en minutos. N01 (azul) es la bodega y los nodos verdes son los destinos.

Para representar los sentidos de circulación separamos los tramos en tres tipos, cada uno con un color distinto en la figura:

Tipo de tramo	Tramos	Arcos	Representación
Doble sentido con el mismo tiempo	49	98	Línea gris con flecha en ambos lados y un solo número
Doble sentido con tiempos distintos	14	28	Dos flechas naranjas curvas, cada una con su tiempo
Un solo sentido	18	18	Flecha roja en el único sentido permitido

Por ejemplo, ir de N04 a N11 toma 9 minutos, pero de N11 a N04 solo 5. Hay calles de un solo sentido como de N01 a N17, de N02 a N18 o de N11 a N13, que no se pueden usar para regresar. En el código guardamos el grafo como una lista de adyacencia (un diccionario `G[origen][destino] = minutos`), así que la asimetría se maneja sola.

Validación de las rutas registradas

Revisamos tramo por tramo las cuatro rutas de ida y sus regresos. Las ocho son válidas, porque todos los tramos existen en el sentido en que se recorren.

Destino	Sentido	Ruta registrada	Tiempos por tramo (min)	Total (min)
N08	Ida	N01, N02, N03, N04, N05, N06, N07, N08	8 + 8 + 9 + 10 + 10 + 7 + 7	59
N08	Regreso	N08, N07, N06, N05, N04, N03, N02, N01	7 + 7 + 10 + 10 + 9 + 8 + 8	59
N19	Ida	N01, N02, N03, N11, N19	8 + 8 + 8 + 9	33
N19	Regreso	N19, N11, N03, N02, N01	9 + 8 + 8 + 8	33
N34	Ida	N01, N09, N17, N25, N33, N34	7 + 9 + 8 + 8 + 10	42
N34	Regreso	N34, N33, N25, N17, N09, N01	10 + 8 + 8 + 9 + 7	42
N40	Ida	N01, N02, ..., N08, N16, N24, N32, N40	59 + 10 + 10 + 6 + 8	93
N40	Regreso	N40, N32, N24, N16, N08, ..., N01	8 + 6 + 10 + 10 + 59	93

Todas las rutas registradas usan solo tramos de doble sentido con el mismo tiempo, por eso la ida y el regreso dan igual. Algo importante es que la ruta a N40 tarda 93 minutos solo de ida, así que ningún pedido a N40 puede llegar a tiempo aunque no tenga que esperar vehículo.

Selección del algoritmo

Investigamos cuatro algoritmos de ruta más corta (Dijkstra, A*, Bellman-Ford y Floyd-Warshall) y además el algoritmo de Yen, que usamos para encontrar rutas alternativas. Implementamos todos en Python para poder compararlos sobre la misma red.

Algoritmo	Cómo funciona	Complejidad	Ventajas para esta red	Limitaciones para esta red
Dijkstra	Va tomando el nodo no visitado con menor distancia y actualiza a sus vecinos	$O((V + E) \log V)$ con heap	Da la ruta óptima si no hay pesos negativos (aquí no hay); funciona igual con arcos dirigidos	Explora en todas las direcciones, no usa información de ubicación
A*	Como Dijkstra, pero ordena por $g(n) + h(n)$, donde h es una estimación de lo que falta	Igual que Dijkstra en el peor caso	Puede revisar menos nodos si la heurística es buena	No tenemos coordenadas, entonces hay que inventar la heurística a partir del grafo
Bellman-Ford	Relaja todos los arcos hasta $V - 1$ veces	$O(V \cdot E)$	Acepta pesos negativos y detecta ciclos negativos	No hay pesos negativos, así que hace trabajo de más
Floyd-Warshall	Programación dinámica con todos los nodos intermedios	$O(V^3)$	Calcula las distancias entre todos los pares de una vez	Solo necesitamos 8 pares, no los 1,560
Yen	Encuentra las k rutas simples más cortas usando Dijkstra varias veces	$O(k \cdot V \cdot (V + E) \log V)$	Sirve para generar las rutas alternativas	No es un algoritmo de ruta óptima por sí solo

Los cuatro algoritmos dieron los mismos tiempos en los ocho pares, lo cual nos sirvió para verificar la implementación. También medimos cuántos nodos revisa cada uno y cuánto tarda (promedio de 300 ejecuciones):

Par	Tiempo óptimo (min)	Nodos expandidos Dijkstra	Nodos expandidos A*	Relajaciones Bellman-Ford	Dijkstra (µs)	A* (µs)	Bellman-Ford (µs)
N01 a N08	51	39	29	432	30	76	67
N08 a N01	59	40	34	1,008	27	92	170
N01 a N19	18	9	5	432	7	56	77
N19 a N01	24	21	7	720	18	60	160
N01 a N34	24	15	7	432	24	102	124
N34 a N01	34	26	8	864	41	103	158
N01 a N40	47	37	18	432	28	98	127
N40 a N01	61	40	39	1,296	53	152	256

Floyd-Warshall tardó unos 2,390 µs en calcular todos los pares.

Heurística usada en A*

Como la matriz no incluye coordenadas, no se puede usar la distancia en línea recta. Definimos h(n) como el número mínimo de tramos que hay de n al destino multiplicado por el tiempo del tramo más corto de toda la red, que es 4 minutos. El número de tramos lo calculamos con una búsqueda en anchura sobre el grafo con los arcos invertidos. Como cualquier camino de s tramos dura al menos 4s minutos, esta heurística nunca sobreestima (es admisible), y por eso A* sigue encontrando la ruta óptima.

El efecto fue que A* revisó menos nodos, hasta 69 % menos en el par N34 a N01 (8 contra 26). Sin embargo, en tiempo real fue entre 2 y 8 veces más lento, porque calcular la heurística cuesta más de lo que se ahorra en una red tan pequeña.

Algoritmo elegido

Elegimos Dijkstra. La red cumple su única condición (todos los tiempos son positivos), siempre da la ruta óptima, fue el más rápido en todos los pares y es fácil volver a correrlo cuando cambia un tramo. Cuando hay un incidente solo hay que recalcular 8 rutas (de la bodega a cada destino y de regreso), lo que toma menos de medio milisegundo.

Validación de alternativas

Para cada destino comparamos la ruta actual con la ruta de Dijkstra y con otras dos alternativas obtenidas con el algoritmo de Yen, tanto de ida como de regreso. En general las rutas actuales no convienen: Dijkstra encontró una ruta más rápida en todas las idas y en tres de los cuatro regresos. El único caso en que la ruta actual ya es la mejor es el regreso desde N08.

Tabla comparativa por destino

Destino	Ida actual (min)	Ida Dijkstra (min)	Regreso actual (min)	Regreso Dijkstra (min)	Ciclo actual (min)	Ciclo Dijkstra (min)	Ahorro por ciclo
N08	59	51	59	59	118	110	8 min (6.8 %)
N19	33	18	33	24	66	42	24 min (36.4 %)
N34	42	24	42	34	84	58	26 min (31.0 %)
N40	93	47	93	61	186	108	78 min (41.9 %)
Promedio	56.75	35.0	56.75	44.5	113.5	79.5	34 min (30.0 %)

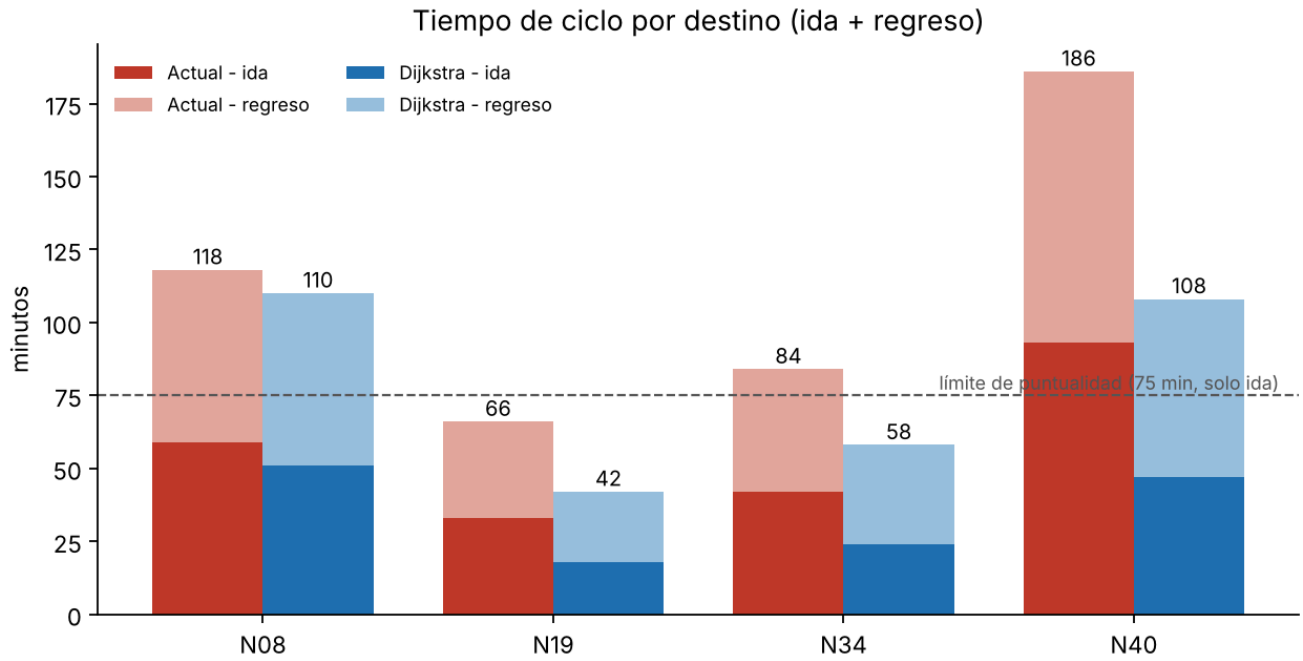


Figura 2. Tiempo de ida y regreso por destino con cada política.

Como los cuatro destinos reciben la misma cantidad de pedidos, el ciclo promedio de un vehículo es el promedio simple de los cuatro ciclos:

- Rutas actuales: (118 + 66 + 84 + 186) / 4 = 113.5 min
- Rutas de Dijkstra: (110 + 42 + 58 + 108) / 4 = 79.5 min

Notamos que con Dijkstra la ida y el regreso casi nunca son el mismo camino. Esto pasa porque la red es asimétrica; por ejemplo, para ir a N19 conviene usar el tramo de N02 a N18, pero ese tramo es de un solo sentido y no sirve para volver.

Destino N08

Sentido	Opción	Ruta	Minutos	Diferencia vs. actual
Ida	Actual	N01, N02, N03, N04, N05, N06, N07, N08	59	
Ida	Dijkstra	N01, N09, N10, N11, N13, N22, N23, N07, N08	51	-8
Ida	Alternativa 2	N01, N02, N03, N11, N13, N22, N23, N07, N08	52	-7

Sentido	Opción	Ruta	Minutos	Diferencia vs. actual
Ida	Alternativa 3	N01, N09, N10, N11, N13, N22, N23, N16, N08	54	-5
Regreso	Actual (igual a Dijkstra)	N08, N07, N06, N05, N04, N03, N02, N01	59	0
Regreso	Alternativa 2	N08, N07, N06, N05, N12, N03, N02, N01	59	0
Regreso	Alternativa 3	N08, N07, N23, N22, N29, N27, N11, N10, N09, N01	62	+3

La ida más rápida pasa por el tramo de N11 a N13, que solo se puede usar en ese sentido. Por eso para el regreso lo mejor sigue siendo la ruta actual, y en este destino solo conviene cambiar la ida.

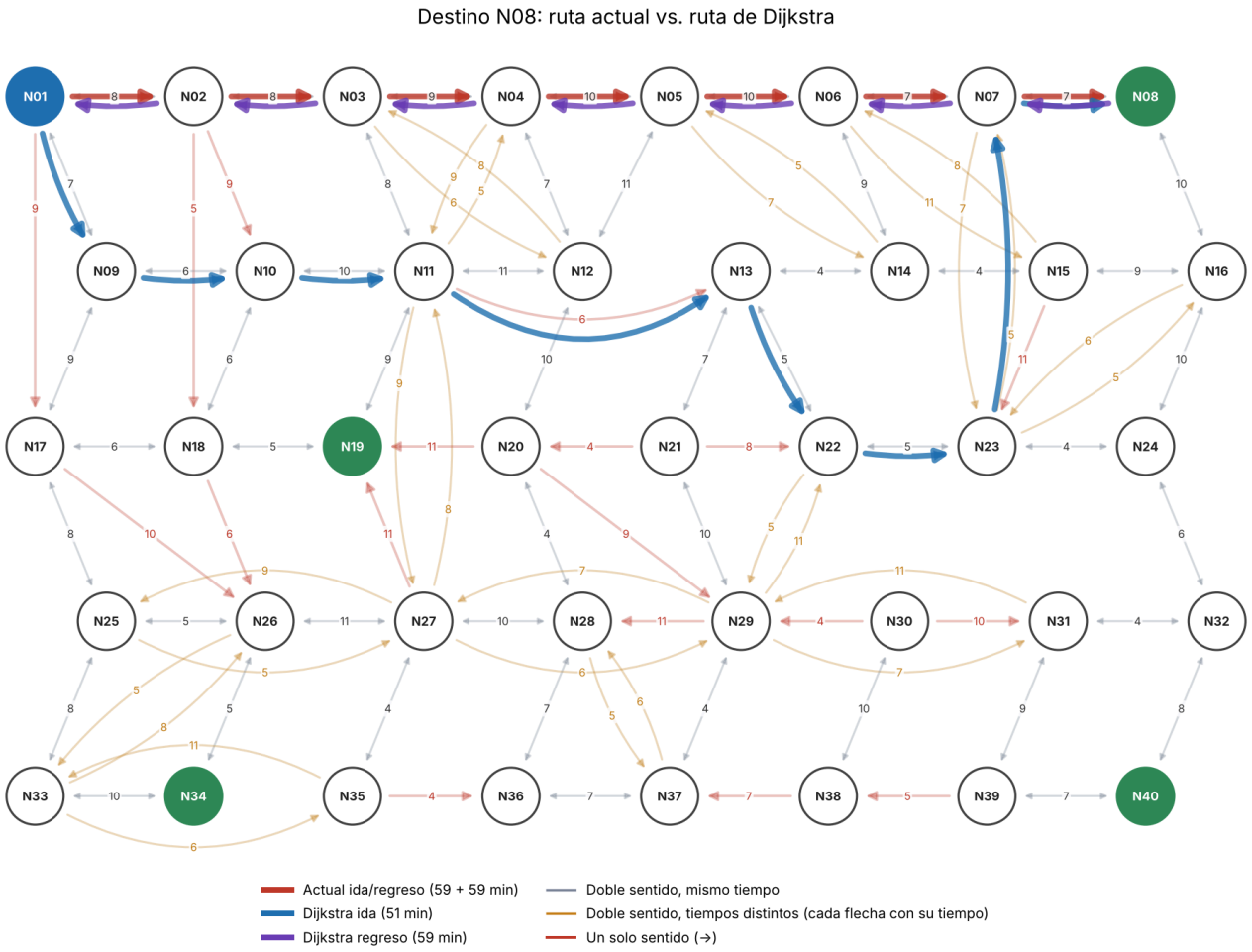


Figura 3. Ruta actual y rutas de Dijkstra para N08.

Destino N19

Sentido	Opción	Ruta	Minutos	Diferencia vs. actual
Ida	Actual	N01, N02, N03, N11, N19	33	
Ida	Dijkstra	N01, N02, N18, N19	18	-15
Ida	Alternativa 2	N01, N17, N18, N19	20	-13
Ida	Alternativa 3	N01, N09, N10, N18, N19	24	-9
Regreso	Actual	N19, N11, N03, N02, N01	33	
Regreso	Dijkstra	N19, N18, N10, N09, N01	24	-9
Regreso	Alternativa 2	N19, N18, N17, N09, N01	27	-6
Regreso	Alternativa 3	N19, N11, N10, N09, N01	32	-1

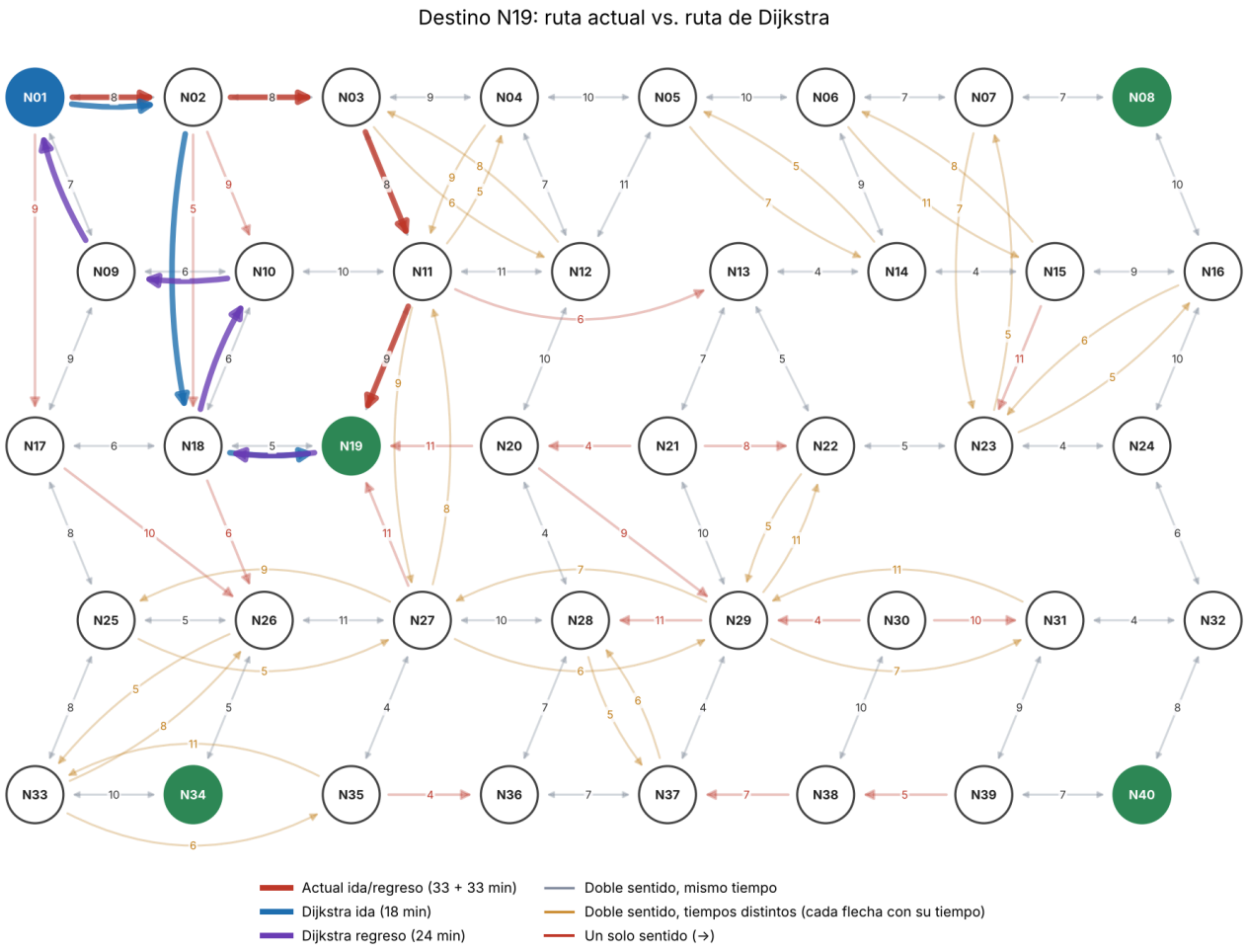


Figura 4. Ruta actual y rutas de Dijkstra para N19.

Destino N34

Sentido	Opción	Ruta	Minutos	Diferencia vs. actual
Ida	Actual	N01, N09, N17, N25, N33, N34	42	
Ida	Dijkstra	N01, N17, N26, N34	24	-18
Ida	Alternativa 2 (empate)	N01, N02, N18, N26, N34	24	-18
Ida	Alternativa 3	N01, N17, N18, N26, N34	26	-16
Regreso	Actual	N34, N33, N25, N17, N09, N01	42	
Regreso	Dijkstra	N34, N26, N25, N17, N09, N01	34	-8
Regreso	Alternativa 2	N34, N26, N33, N25, N17, N09, N01	42	0
Regreso	Alternativa 3	N34, N26, N25, N17, N18, N10, N09, N01	43	+1

En la ida hay un empate de 24 minutos entre dos rutas. Dijkstra se queda con la primera que encuentra, y la otra puede servir de respaldo si hay tráfico entre N01 y N17.

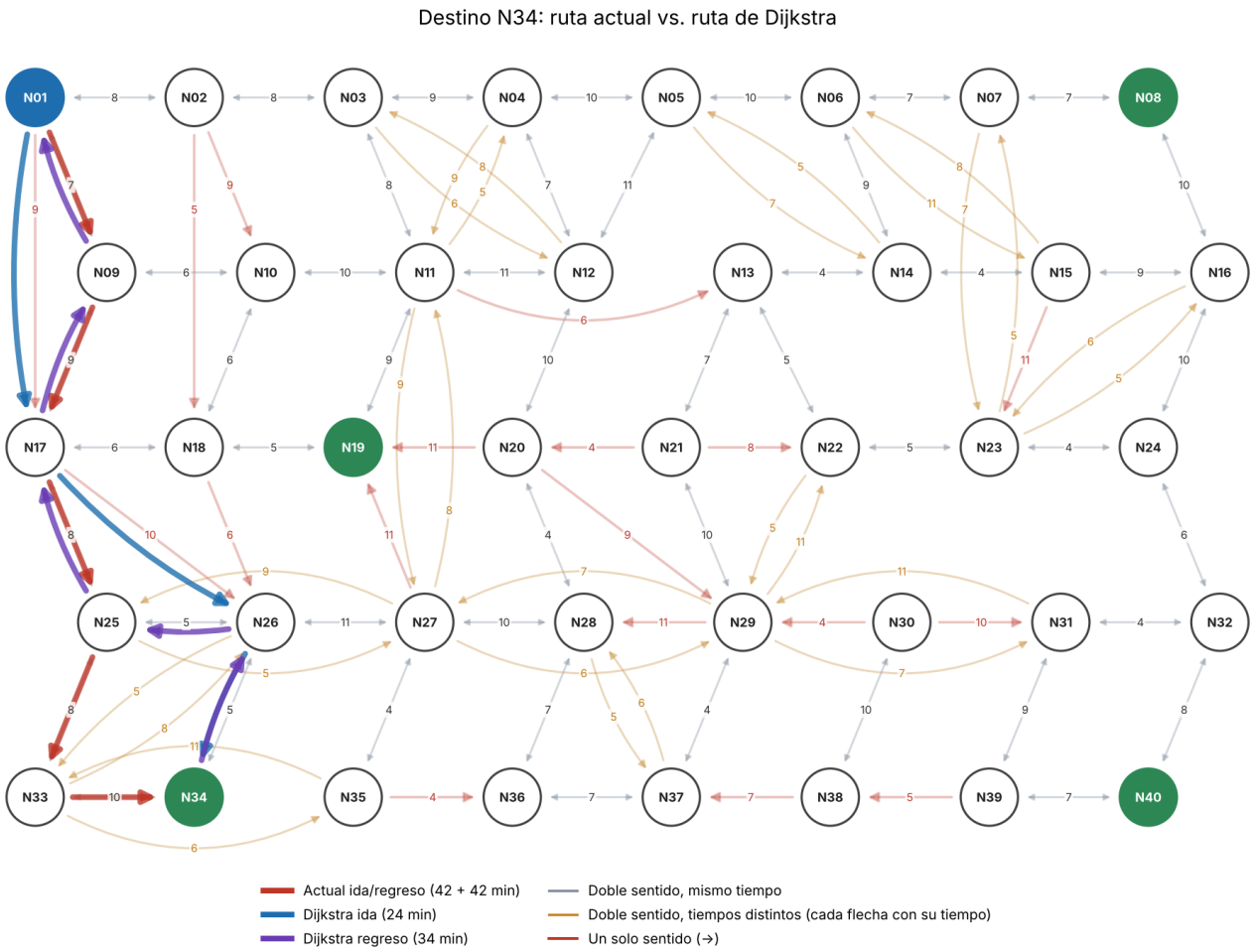


Figura 5. Ruta actual y rutas de Dijkstra para N34.

Destino N40

Sentido	Opción	Ruta	Minutos	Diferencia vs. actual
Ida	Actual	N01, N02, ..., N08, N16, N24, N32, N40	93	
Ida	Dijkstra	N01, N17, N25, N27, N29, N31, N32, N40	47	-46
Ida	Alternativa 2	N01, N17, N25, N27, N29, N31, N39, N40	51	-42
Ida	Alternativa 3	N01, N09, N17, N25, N27, N29, N31, N32, N40	54	-39

Sentido	Opción	Ruta	Minutos	Diferencia vs. actual
Regreso	Actual	N40, N32, N24, N16, N08, ..., N01	93	
Regreso	Dijkstra	N40, N32, N31, N29, N27, N11, N10, N09, N01	61	-32
Regreso	Alternativa 2 (empate)	N40, N39, N38, N37, N29, N27, N11, N10, N09, N01	61	-32
Regreso	Alternativa 3	N40, N32, N31, N29, N27, N11, N03, N02, N01	62	-31

Aquí está la mayor diferencia. La ruta actual va primero hasta N08 y luego baja por toda la columna de la derecha, mientras que la de Dijkstra cruza en diagonal por N25, N27, N29 y N31 y tarda la mitad (47 contra 93 minutos). Con esta ruta un pedido a N40 que no espera llega en 47 minutos y sí puede ser puntual.

Destino N40: ruta actual vs. ruta de Dijkstra

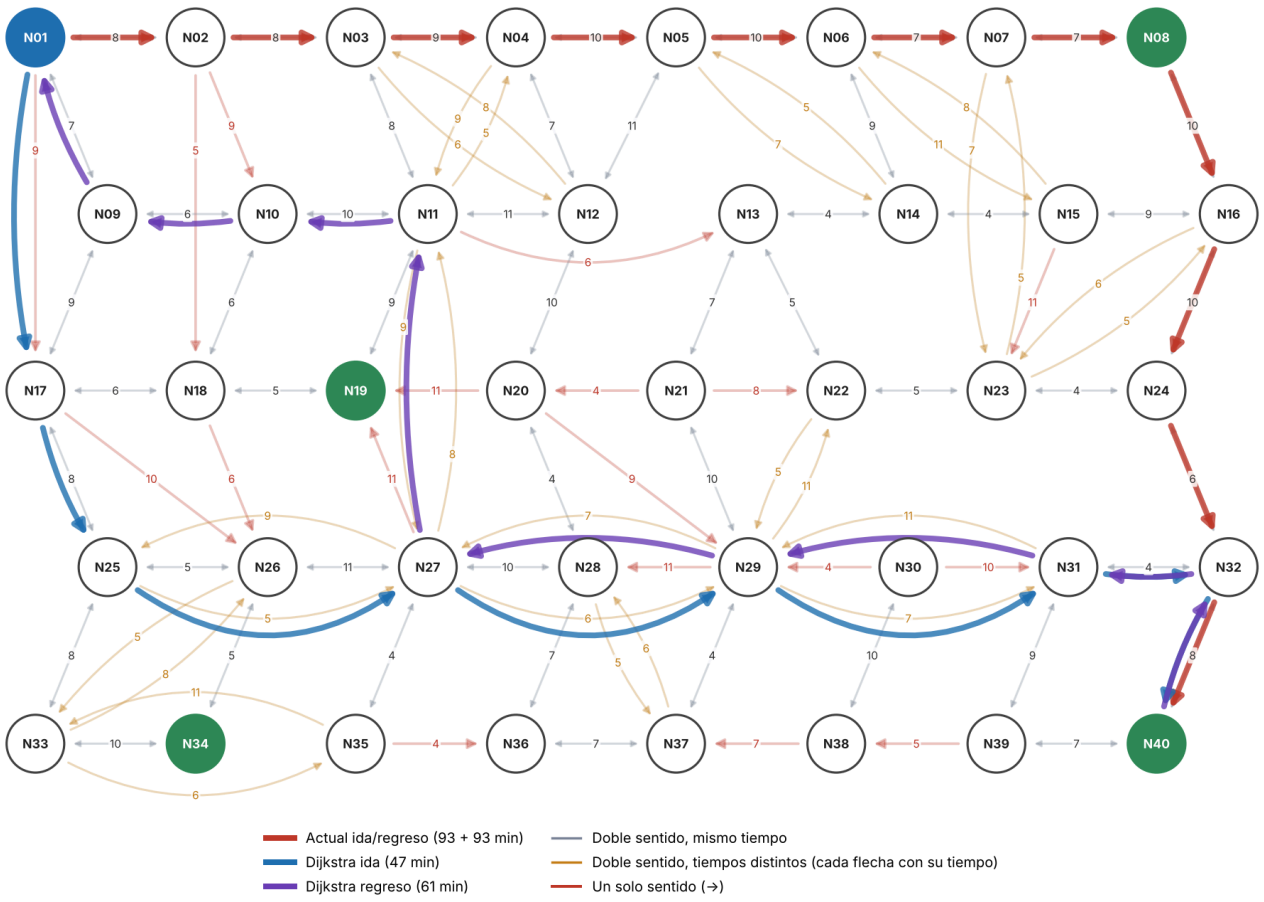


Figura 6. Ruta actual y rutas de Dijkstra para N40.

Descripción del modelo de simulación

Modelamos la operación como una cola con varios servidores, donde cada vehículo es un servidor y los pedidos se atienden por orden de llegada. El tiempo que un vehículo está ocupado con un pedido es todo su ciclo: el viaje de ida, la descarga y el regreso a la bodega. Este tiempo depende del destino y de la política de rutas que se use.

Generación de pedidos y destinos

Los tiempos entre llegadas los generamos con una distribución exponencial de media 60/λ minutos, es decir, 15 minutos en un día normal, 10 en Semana Santa y 6 en Navidad. Vamos sumando estos tiempos desde el minuto 0 y nos quedamos con los pedidos que caen antes del minuto 480 (fin de las 8 horas). Así la cantidad de pedidos del día es aleatoria, con un promedio de 32, 48 y 80 pedidos según el escenario.

El destino de cada pedido se escoge con una distribución uniforme entre los cuatro destinos. Para el análisis de sensibilidad, a cada pedido también se le asignan dos factores de tráfico (uno para la ida y otro para el regreso) con distribución lognormal de media 1, que multiplican el tiempo de la ruta. Usamos `numpy` con un generador distinto para las llegadas, los destinos y el tráfico, todos derivados de la misma semilla, para que cambiar una parte no altere las otras.

Eventos

La simulación tiene cuatro tipos de evento:

1. Llegada: el pedido entra a la cola.
2. Salida: si hay un vehículo libre en N01, sale con el primer pedido de la cola. La espera del pedido es el tiempo entre su llegada y su salida.
3. Entrega: ocurre cuando termina el viaje de ida más la descarga. El pedido cuenta como puntual si se entrega en 75 minutos o menos desde que llegó.
4. Regreso: el vehículo vuelve a N01, queda libre y se revisa si hay otro pedido esperando.

Los eventos futuros se guardan en un heap ordenado por tiempo. Si un regreso y una llegada ocurren en el mismo instante, primero se procesa el regreso. La simulación termina cuando ya no quedan eventos, o sea cuando el último vehículo regresa, aunque eso pase después de las 8 horas.

Políticas comparadas

La política actual usa las rutas de las hojas de despacho y regresa por el mismo camino. La política de Dijkstra usa la ruta más corta de N01 al destino para ir y la ruta más corta del destino a N01 para regresar.

Para que la comparación sea justa usamos números aleatorios comunes: con cada semilla generamos una sola lista de pedidos y la simulamos con las dos políticas. De esta forma las dos corridas de cada pareja solo cambian en las rutas, y podemos comparar los resultados con una prueba t pareada.

Métricas

Métrica	Cómo se calcula
Entregas puntuales	Pedidos con tiempo desde la solicitud hasta la entrega menor o igual a 75 min
Espera promedio	Promedio de (salida - llegada)
Tiempo solicitud-entrega	Promedio de (entrega - llegada), incluye espera y viaje de ida
Utilización	Minutos que los vehículos pasan en ruta / (número de vehículos x duración total de la operación)
Tiempo extra	Minuto en que regresa el último vehículo menos 480

Vale aclarar que el tiempo extra nunca es cero, porque aunque no haya cola el último pedido del día igual necesita su viaje completo.

Verificación

Para verificar el simulador comparamos los tiempos de entrega con un cálculo independiente (asignando cada pedido al vehículo que se libera primero) y la diferencia fue 0. Los tiempos de las rutas también los comprobamos con Floyd-Warshall y con la librería `networkx`.

Aplicación

La aplicación está hecha con Streamlit y se ejecuta con `streamlit run app.py`. En el panel izquierdo se puede escoger el escenario (normal, Semana Santa, Navidad o personalizado) y cambiar los pedidos por hora, el número de vehículos, la duración de la recepción de pedidos, la semilla, el tiempo de descarga y la variabilidad del tráfico.

La aplicación tiene cuatro pestañas. En Red y rutas hay una tabla con los 144 arcos donde se puede cambiar el tiempo de un tramo (congestión) o marcarlo como cerrado; abajo se muestran las rutas de las dos políticas, las alternativas y el mapa. En Simulación se corre un día con ambas políticas y se ven las métricas, la cola en el tiempo, un histograma de los tiempos de entrega, un diagrama de Gantt de los vehículos y la bitácora de eventos. En Experimentos se hacen varias réplicas pareadas, y en Tamaño de flota se prueba un rango de vehículos.

Actualización de rutas cuando cambia la red

Cada vez que se modifica un tramo, la aplicación vuelve a armar el grafo y corre Dijkstra otra vez para los 8 pares. En la política actual, si un tramo de la ruta quedó cerrado, el conductor sigue su ruta hasta el punto anterior al cierre y desde ahí toma el camino más corto (la app lo marca como desvío). Si algún destino queda sin conexión, la app avisa y no simula. Probamos estos incidentes:

Incidente	Efecto en las rutas actuales	Efecto en las rutas de Dijkstra
Cierre de N17 a N25	Ida a N34 se desvía en N17 por N26: 42 a 31 min	Ida a N40 pasa por N26: 47 a 54 min
Cierre de N03 a N04	Ida a N08 pasa por N11, N13 y N22: 59 a 52 min; ida a N40: 93 a 58 min	No cambia
Congestión de N02 a N18 (5 a 15 min)	No cambia	Ida a N19 por N17 y N18: 18 a 20 min
Cierre de N29 a N31 y de N27 a N29	No cambia	Ida a N40 por N13, N22, N23 y N24: 47 a 57 min

En dos de los cierres el desvío de la política actual terminó siendo más rápido que la ruta original, lo que confirma que las rutas de las hojas no estaban optimizadas. Con Dijkstra, el peor incidente que probamos solo aumentó el ciclo promedio en 2.5 minutos y la puntualidad en día normal se mantuvo en 97 %.

Resultados

Hicimos 10 réplicas por escenario (semillas 1 a 10) con 8 vehículos. En cada réplica las dos políticas atendieron exactamente los mismos pedidos. La siguiente tabla muestra el promedio de las 10 réplicas y el semiancho del intervalo de confianza del 95 %.

Escenario	Política	Pedidos	Entregas puntuales	% puntual	Espera prom. (min)	Solicitud-entrega (min)	Utilización (%)	Tiempo extra (min)
Normal (4/h)	Actual	31.9 ± 4.5	17.2 ± 3.9	57.0 ± 16.6	19.4 ± 16.5	75.0 ± 16.3	66.8 ± 7.4	179.1 ± 32.2
Normal (4/h)	Dijkstra	31.9 ± 4.5	31.1 ± 4.7	97.6 ± 5.3	3.0 ± 4.5	37.1 ± 4.8	54.6 ± 7.9	86.8 ± 17.7
Semana Santa (6/h)	Actual	46.9 ± 6.2	12.3 ± 4.1	28.6 ± 14.4	82.8 ± 32.0	138.4 ± 33.0	81.5 ± 6.7	314.4 ± 66.0
Semana Santa (6/h)	Dijkstra	46.9 ± 6.2	33.9 ± 6.6	75.3 ± 17.7	21.6 ± 13.0	55.3 ± 13.0	75.6 ± 7.5	112.7 ± 32.1
Navidad (10/h)	Actual	80.6 ± 7.4	8.0 ± 2.6	10.4 ± 4.4	285.7 ± 57.5	341.9 ± 58.4	92.3 ± 1.5	748.1 ± 121.4
Navidad (10/h)	Dijkstra	80.6 ± 7.4	19.0 ± 5.8	24.6 ± 9.9	136.6 ± 35.8	171.2 ± 35.9	90.5 ± 2.2	393.3 ± 71.9

En día normal y en Semana Santa las rutas de Dijkstra mejoran mucho la puntualidad (de 57 % a 98 % y de 29 % a 75 %). En Navidad mejora un poco, pero con ninguna de las dos políticas se logra una operación aceptable.

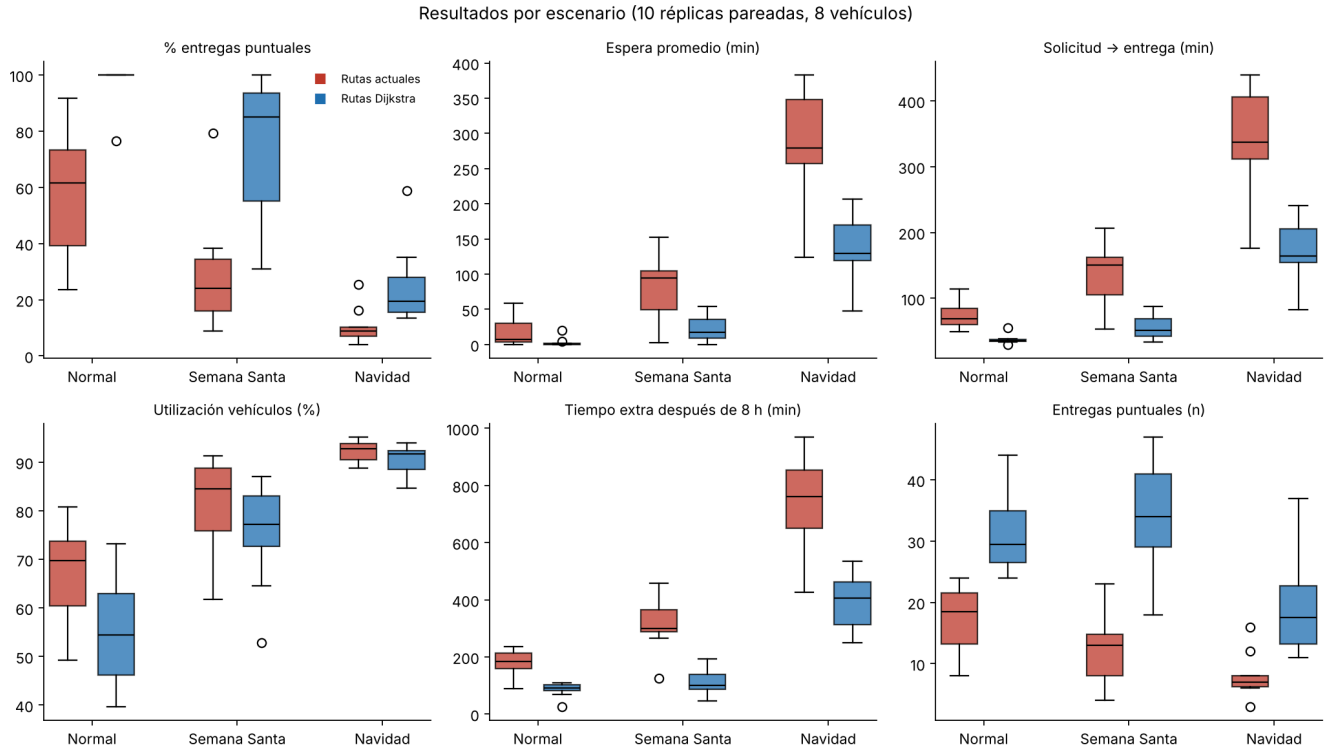


Figura 7. Distribución de las métricas en las 10 réplicas de cada escenario (rojo: rutas actuales, azul: Dijkstra).

Comparación pareada

Con las parejas de corridas calculamos la diferencia Dijkstra menos Actual y su intervalo de confianza del 95 %:

Escenario	Entregas puntuales	Espera prom. (min)	Solicitud-entrega (min)	Utilización (puntos)	Tiempo extra (min)
Normal	+13.9 [7.3; 20.5]	-16.4 [-29.9; -2.9]	-37.9 [-51.4; -24.3]	-12.2 [-14.9; -9.5]	-92.3 [-125.0; -59.5]
Semana Santa	+21.6 [14.5; 28.7]	-61.2 [-82.5; -40.0]	-83.1 [-105.5; -60.6]	-5.9 [-7.7; -4.2]	-201.7 [-239.0; -164.5]
Navidad	+11.0 [7.3; 14.7]	-149.1 [-173.0; -125.2]	-170.7 [-195.7; -145.7]	-1.8 [-4.0; 0.3]	-354.8 [-409.3; -300.3]

Todas las diferencias fueron significativas con la prueba t pareada ($p < 0.03$), excepto la utilización en Navidad ($p = 0.08$), porque ahí las dos flotas están saturadas. También se ve que los intervalos de las diferencias son más angostos que los de cada política por separado, que es justamente la ventaja de usar los mismos pedidos en las dos corridas.

Evolución de la cola

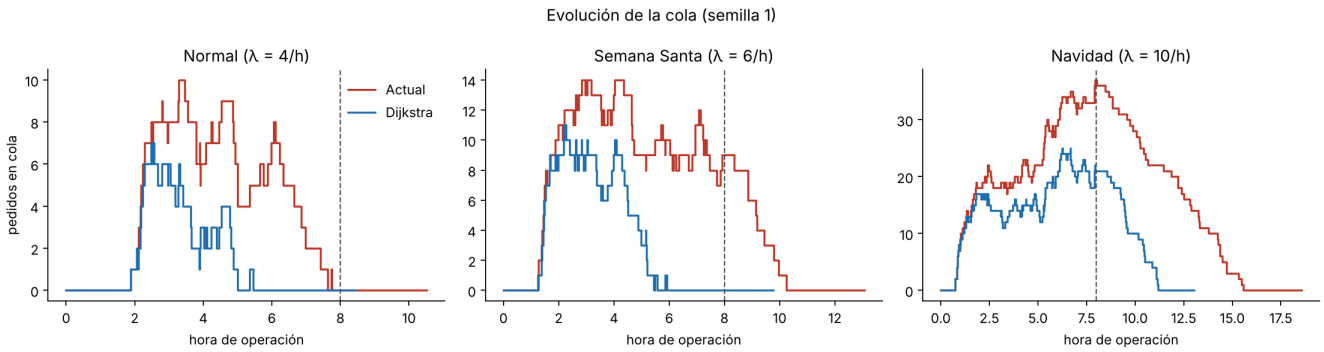


Figura 8. Pedidos en cola durante el día para la semilla 1.

En Navidad la cola crece casi en línea recta durante las 8 horas con las dos políticas, porque llegan pedidos más rápido de lo que la flota puede atender. En Semana Santa con Dijkstra la cola sube y baja, lo que indica que el sistema está trabajando al límite.

Número de vehículos

Repetimos las 10 réplicas variando la flota de 4 a 24 vehículos.

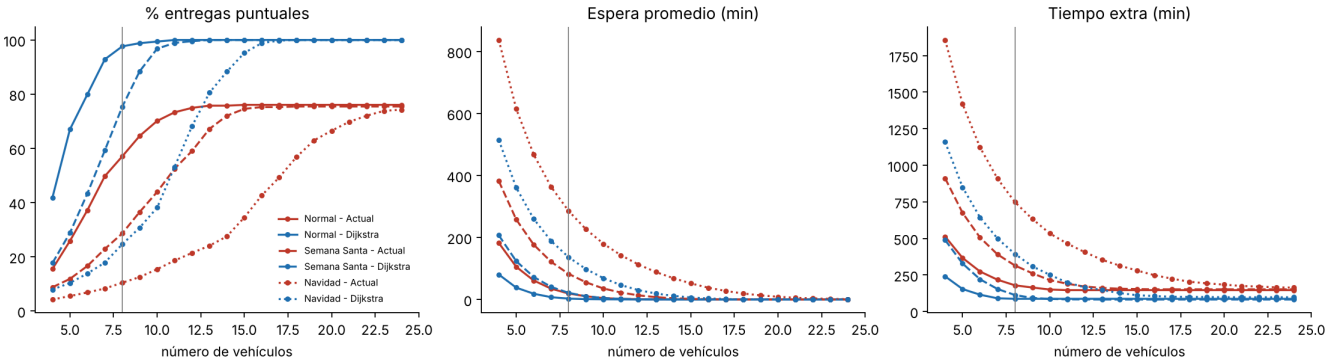


Figura 9. Puntualidad, espera y tiempo extra según el número de vehículos. La línea vertical marca la flota actual de 8.

Escenario	Vehículos para 95 % puntual (Dijkstra)	Vehículos para 99 % puntual (Dijkstra)	Máximo alcanzado con rutas actuales
Normal	8	10	76 % (desde 15 vehículos)
Semana Santa	10	12	75 % (desde 16 vehículos)
Navidad	15	17	74 % (con 24 vehículos)

Con las rutas actuales la puntualidad no pasa de 76 % aunque se agreguen muchos vehículos. Esto se debe a que los pedidos a N40, que son más o menos la cuarta parte, tardan 93 minutos solo de ida. Más vehículos eliminan la espera, pero no acortan el viaje.

Análisis de sensibilidad

Repetimos los experimentos agregando 5 minutos de descarga y variabilidad en el tráfico (coeficiente de variación de 0.15):

Escenario	% puntual Actual	% puntual Dijkstra	Espera prom. Actual (min)	Espera prom. Dijkstra (min)
Normal	54.2	93.5	23.8	4.5
Semana Santa	26.4	67.8	95.6	29.5
Navidad	9.1	20.1	311.4	159.8

La puntualidad baja entre 1 y 8 puntos, pero las conclusiones no cambian: Dijkstra sigue siendo mucho mejor en los tres escenarios.

Discusión

Aporte del cambio de rutas

Cambiar las rutas tiene un efecto parecido a comprar más vehículos, pero sin costo. En Semana Santa, con 8 vehículos y rutas de Dijkstra se logra 75 % de puntualidad, mientras que con las rutas actuales harían falta 16 vehículos para llegar a lo mismo (75.2 %). En

Navidad pasa algo similar: 8 vehículos con Dijkstra (24.6 %) rinden como 13 vehículos con las rutas actuales (24.1 %). En un día normal la diferencia es todavía más clara, porque con las rutas actuales nunca se pasa de 76 % y con Dijkstra los 8 vehículos alcanzan 98 %.

Creemos que la mejora viene de dos lados. Primero, el viaje de ida es más corto (35 minutos en promedio contra 56.75), y eso hace que los pedidos a N40 puedan llegar a tiempo. Segundo, como el ciclo completo es 30 % más corto, los vehículos quedan libres antes y la espera de todos los pedidos baja (en día normal, de 19.4 a 3 minutos).

Al principio nos pareció raro que la utilización de los vehículos bajara con Dijkstra (de 66.8 % a 54.6 % en día normal), pero tiene sentido: se entregan los mismos pedidos con menos minutos de viaje. Ese tiempo libre es lo que permite absorber los picos de demanda.

Efecto de aumentar la demanda

Para analizar la carga de la flota usamos el factor de utilización de una cola con c servidores:

```
\rho = \frac{\lambda \cdot E[\text{ciclo}]}{60 \cdot c}
```

donde λ es la cantidad de pedidos por hora, E[ciclo] es el ciclo promedio en minutos y c es el número de vehículos. Si ρ es mayor o igual a 1, la flota no da abasto y la cola crece durante todo el día.

Escenario	ρ Actual (8 veh.)	ρ Dijkstra (8 veh.)	Vehículos mínimos para ρ < 1 (Actual)	Vehículos mínimos para ρ < 1 (Dijkstra)
Normal (4/h)	0.95	0.66	8	6
Semana Santa (6/h)	1.42	0.99	12	8
Navidad (10/h)	2.37	1.66	19	14

El aumento de la demanda no tiene un efecto lineal. Al pasar de día normal a Semana Santa hay 50 % más pedidos, pero la espera promedio con Dijkstra pasa de 3 a 21.6 minutos (unas 7 veces más). En Navidad hay 150 % más pedidos y la espera llega a 136.6 minutos, y el último vehículo regresa unas 6.5 horas después de que se cierra la recepción. Esto coincide con lo que se espera en teoría de colas: cuando ρ se acerca a 1, la espera crece muy rápido.

Cuándo la flota se vuelve una limitación

Según los resultados, la flota empieza a ser un problema cuando ρ pasa de más o menos 0.8. Con las rutas de Dijkstra, en un día normal los 8 vehículos son suficientes (ρ = 0.66), e incluso con 7 se tendría 93 % de puntualidad. En Semana Santa los 8 vehículos están justo en el límite (ρ = 0.99) y los resultados varían bastante de un día a otro (entre 31 % y 100 % en las réplicas), así que se recomiendan 10. En Navidad la flota sí es la limitación principal (ρ = 1.66) y se necesitarían 15 vehículos para llegar a 95 %.

Con las rutas actuales la flota ya está casi saturada en un día normal (ρ = 0.95). Esto explicaría por qué la gerencia ya notaba problemas aunque no fuera temporada alta.

Sobre el tiempo extra, hay que tomar en cuenta que nunca llega a cero, porque el último pedido que entra cerca del minuto 480 necesita su ciclo completo. Por eso el mínimo es de unos 80 a 90 minutos con Dijkstra y de unos 145 minutos con las rutas actuales.

Limitaciones de la simulación

Nuestro modelo deja fuera varios aspectos de la operación real. Los tiempos de viaje son fijos y no consideran horas pico, semáforos ni accidentes (solo el factor de tráfico opcional, que es igual para todo el viaje). Cada vehículo lleva un solo pedido, cuando en la realidad se podrían juntar pedidos para la misma zona. También asumimos que los pedidos llegan con la misma intensidad durante todo el día y que los cuatro destinos reciben la misma cantidad, lo cual probablemente no pasa siempre.

Además, no tomamos en cuenta el tiempo de carga en la bodega, el tiempo para estacionarse o atender al cliente, los turnos y descansos de los conductores, las fallas de los vehículos ni la posibilidad de dar prioridad a pedidos urgentes. Por último, las rutas de Dijkstra se calcularon con tiempos promedio, y en la práctica la mejor ruta podría cambiar según el tráfico del momento.

Conclusiones

1. La red tiene 40 nodos y 144 arcos dirigidos, con 14 tramos que tienen tiempos distintos en cada sentido y 18 de un solo sentido. Las cuatro rutas de las hojas de despacho se pueden recorrer de ida y de regreso, así que son válidas, pero no son las más rápidas.
2. Con Dijkstra encontramos rutas mejores en 7 de los 8 casos, y el ciclo promedio bajó de 113.5 a 79.5 minutos (30 % menos). El cambio más importante es el de N40, que pasa de 93 a 47 minutos de ida y deja de ser un destino donde nunca se cumple el tiempo. La única ruta actual que vale la pena mantener es el regreso desde N08.

3. En la simulación con 8 vehículos, usar las rutas de Dijkstra aumentó las entregas puntuales en 13.9 pedidos por día en día normal, 21.6 en Semana Santa y 11.0 en Navidad. La espera promedio bajó entre 16 y 149 minutos y la operación terminó entre 1.5 y 5.9 horas antes.
4. Con las rutas de Dijkstra se necesitan 8 vehículos en día normal, 10 en Semana Santa y 15 en Navidad para tener al menos 95 % de entregas puntuales. Con las rutas actuales no se pasa de 76 % sin importar cuántos vehículos haya. En los incidentes que probamos, recalcular con Dijkstra aumentó una ruta como máximo en 10 minutos.

Reflexión final

Para la empresa, la medida más fácil y barata sería actualizar las hojas de despacho con las rutas de Dijkstra. Solo con eso la flota actual queda con margen en días normales y rinde en Semana Santa como si tuviera el doble de vehículos. Para Navidad el cambio de rutas no es suficiente, y habría que conseguir entre 7 y 9 vehículos más (por ejemplo alquilados) o buscar otras formas de acortar los ciclos, como juntar pedidos de la misma zona. La aplicación le permite a la gerente repetir este análisis si cambia la red o la demanda.

Sin embargo, hay que tener cuidado al aplicar estos resultados directamente. El modelo depende de que los tiempos de la matriz representen bien la realidad y de que la demanda sea pareja durante el día y entre zonas. Antes de tomar una decisión como comprar vehículos, sería recomendable medir los tiempos reales de cada tramo en distintas horas, registrar cuántos pedidos llegan por hora y a qué zona, y comparar el modelo con datos históricos de la operación. Por eso los números de vehículos que recomendamos deben tomarse como una guía y no como un valor exacto.

Referencias

Banks, J., Carson, J. S., Nelson, B. L., & Nicol, D. M. (2010). *Discrete-event system simulation* (5th ed.). Pearson.

Bellman, R. (1958). On a routing problem. *Quarterly of Applied Mathematics, 16*(1), 87–90.

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to algorithms* (4th ed.). MIT Press.

Dijkstra, E. W. (1959). A note on two problems in connexion with graphs. *Numerische Mathematik, 1*(1), 269–271.

Floyd, R. W. (1962). Algorithm 97: Shortest path. *Communications of the ACM, 5*(6), 345.

Hart, P. E., Nilsson, N. J., & Raphael, B. (1968). A formal basis for the heuristic determination of minimum cost paths. *IEEE Transactions on Systems Science and Cybernetics, 4*(2), 100–107.

Law, A. M. (2015). *Simulation modeling and analysis* (5th ed.). McGraw-Hill Education.

Ross, S. M. (2014). *Introduction to probability models* (11th ed.). Academic Press.

Yen, J. Y. (1971). Finding the k shortest loopless paths in a network. *Management Science, 17*(11), 712–716.

Anexo: archivos del código

Archivo	Contenido
`red.py`	Lectura de la matriz, validación de rutas y algoritmos (Dijkstra, A*, Bellman-Ford, Floyd-Warshall y Yen)
`simulacion.py`	Simulación de eventos discretos y políticas de rutas
`visual.py`	Dibujo del grafo y de las rutas
`experimentos.py`	Genera las tablas y figuras del informe (`python experimentos.py`)
`app.py`	Aplicación (`streamlit run app.py`)